

Part II: Graph Algorithms

Lecture 8: Kruskal's MST Algorithm

Computer Science and Technology
United International College

Objective and Outline

Objective: Discuss another algorithm for the MST problem, i.e. Kruskal's algorithm.

Reference: Chapters 21 & 23 of CLRS

1 Kruskal's MST algorithm

- The idea
- The algorithm
- Correctness

2 The Disjoint Set Union-Find data structure

- The basic implementation
- An improvement

Recalling the Generic Algorithm

- Start with an **empty** graph $(V, \emptyset)$.
- Try to **add** one edge at a time, always making sure that the resulting graph remains acyclic.
- If we are sure that every time the resulting graph is always a **subgraph** of one minimum spanning tree, we are done.

Lemma

Let

- $G = (V, E)$ be a connected, undirected graph of a weight function $w : E \rightarrow \mathbb{R}$,
- A be a subset of E such that A is included in one minimum spanning tree for G ,
- $(S, V \setminus S)$ be **an arbitrary** cut of G which respects A , and
- (u, v) be a light edge crossing the cut $(S, V \setminus S)$;

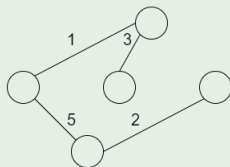
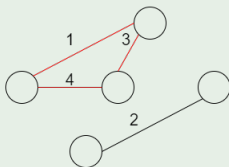
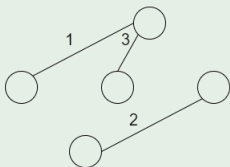
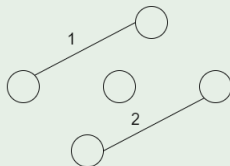
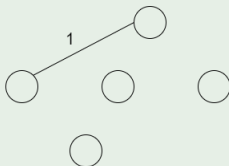
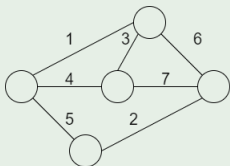
*then, edge (u, v) is **safe** for A .*

Idea of Kruskal's Algorithm

- The **Kruskal's Algorithm** is based directly on the generic algorithm.
- Unlike Prim's algorithm, which grows one tree, Kruskal's algorithm grows a **collection** of trees (a **forest**).
- Initially, trees of the forest are the **vertices** (no edges).
- In each step add the **cheapest** edge that does not create a **cycle**.
- Continue until the forest "merge to" a single tree.

Example

Example



- 1 Kruskal's MST algorithm
 - The idea
 - The algorithm
 - Correctness
- 2 The Disjoint Set Union-Find data structure
 - The basic implementation
 - An improvement

Outline of Kruskal's Algorithm

- 1 Set $A = \emptyset$ and $F = E$, the set of all edges.
- 2 Choose an edge e in F of **minimum** weight, and check whether adding e to A creates a **cycle**.
 - If “yes”, remove e from F .
 - If “no”, move e from F to A .
- 3 If $F = \emptyset$, stop and output the minimal spanning tree (V, A) . Otherwise go to Step 2.

Remark: Will see later, after each step, (V, A) is a subgraph of a MST.

Questions

- How does algorithm **choose** edge $e \in F$ with minimum weight?
- How does algorithm **check** whether adding e to A creates a cycle?

How to Choose the Edge of Least Weight?

Questions

How does algorithm **choose** edge $e \in F$ with minimum weight?

Answer:

- Start by **sorting** edges in E in order of increasing weight.
- Walk through the edges in this order.
- Once edge e causes a cycle, the edge e can be thrown away.

How to Check for Cycles?

Observations:

- At each step of the outlined algorithm, (V, A) is acyclic so it is a forest.
- If u and v are in the same tree, then adding edge (u, v) to A creates a cycle.
- If u and v are not in the same tree, then adding edge (u, v) to A does not create a cycle.

Questions

How to test whether u and v are in the same tree?

High-Level Answer: Use a **disjoint-set data structure**

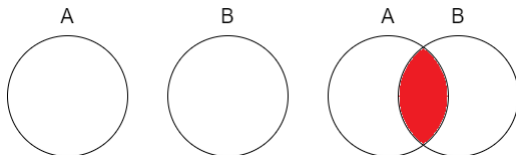
- Vertices in a tree are considered to be in same **set**.
- Test if **$\text{Find-Set}(u) = \text{Find-Set}(v)$** ?

Low-Level Answer:

- The **UNION-FIND** data structure implements this

The UNION-FIND Data Structure

UNION-FIND supports three operations on collections of **disjoint sets**.



Let n be the size of the universe.

- 1 **Create-Set(u)**: Create a set containing the single element u . $\mathcal{O}(1)$
- 2 **Find-Set(u)**: Find the set containing the element u . $\mathcal{O}(\log n)$
- 3 **Union(u, v)**: Merge the sets respectively containing u and v into a common set. $\mathcal{O}(\log n)$

For now we treat UNION-FIND as a black box. The implementation will be given later.

Kruskal's Algorithm: Details

Algorithm 1 Kruskal(G, w)

```
1: sort( $E$ )    // After sorting  $E = \{(u_1, v_1), (u_2, v_2), \dots, (u_e, v_e)\}$ ,  $\mathcal{O}(e \log e)$ 
2:  $A = \{\}$ 
3: for all  $u \in V$  do                                //  $\mathcal{O}(n)$ 
4:   CREATE-SET( $u$ )
5: end for
6: for  $e_i = (u_i, v_i)$  from 1 to  $e$  do                //  $\mathcal{O}(e \log n)$ 
7:   if FIND-SET( $u_i$ )  $\neq$  FIND-SET( $v_i$ ) then
8:     add  $(u_i, v_i)$  to  $A$ 
9:     UNION( $u_i, v_i$ )
10:  end if
11: end for
12: return  $A$ 
```

Remark: With a proper implementation of UNION-FIND, Kruskal's algorithm runs in

$$\mathcal{O}(e \log e) + \mathcal{O}(n) + \mathcal{O}(e \log n) = \mathcal{O}(e \log e)$$

- 1 Kruskal's MST algorithm
 - The idea
 - The algorithm
 - Correctness

- 2 The Disjoint Set Union-Find data structure
 - The basic implementation
 - An improvement

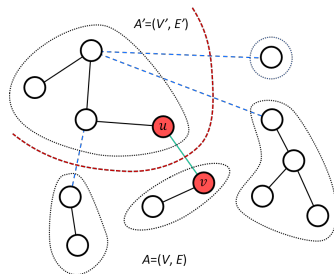
Why Kruskal's Algorithm is Correct? I

Let A be the edge set selected by Kruskal's Algorithm (at the beginning of each iteration), (u, v) be the edge added to A (in each iteration). In other words, we need to show that

- 1 there is a cut which **respects** A , and
- 2 (u, v) is the **light** edge of a cut crossing the cut.

Why Kruskal's Algorithm is Correct? II

- Let $A' = (V', E')$ denote the **tree** of the **forest** A that contains u .
- The partition $(V', V \setminus V')$ forms a cut.
- There is no edge in A crosses this cut, so the cut **respects** A .
- Since adding (u, v) to A' does not produce a cycle, (u, v) **crosses** the cut.
- Moreover, if (u, v) is currently the smallest weighted edge, (u, v) is the **light edge** crossing the cut.



The **red dashed line** is the cut $(V', V \setminus V')$ respect to A . Each dotted ellipse is a tree in A .

- 1 Kruskal's MST algorithm
 - The idea
 - The algorithm
 - Correctness

- 2 The Disjoint Set Union-Find data structure
 - The basic implementation
 - An improvement

Disjoint Set Union-Find

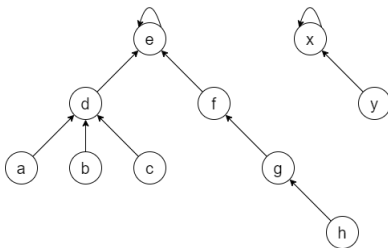
A **disjoint set Union-Find** data structure supports three operation on x and y .

- ① **Create-Set(x)** creates a set containing the single element x .
- ② **Find-Set(x)** finds the set containing the element x .
- ③ **Union(x, y)**
 - merges the set containing x , with the other set containing y to a single set.
 - After this operation, we have $\text{Find-Set}(x) = \text{Find-Set}(y)$.

- 1 Kruskal's MST algorithm
 - The idea
 - The algorithm
 - Correctness

- 2 The Disjoint Set Union-Find data structure
 - The basic implementation
 - An improvement

Up-Tree Implementation



- Every item is in a **tree**.
- The **root** of the tree is the **representative** item of all items in that tree.
 - The root of the tree represents the whole set of elements.
- In this up-tree implementation, every node (except the root) has a pointer pointing to its **parent**.
 - The root element points to itself.

Create-Set(x) and Find-Set(x)

Algorithm 2 Create-Set(x)

1: $x.parent = x$

Easy!

Algorithm 3 Find-Set(x)

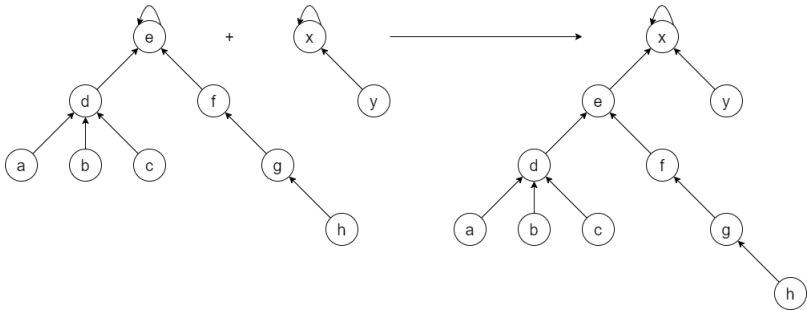
1: **while** $x \neq x.parent$ **do**
2: $x = x.parent$
3: **end while**
4: **return** x

Also easy!

- simply trace the parent point until we hit the root, then return the root element.

Union(x, y)

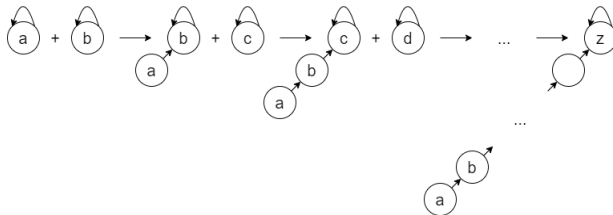
A **naïve solution** can be putting the parent pointer of the representation of x pointing to the representation of y .



Question

Is it a good idea?

Problem



The data structure may become a **linked-list** at the end! Hence the naïve solution is not efficient.

Question

Can we do better?

Simple trick (**Union by height**):

- When we union two trees together, we always make the root of **taller** tree the parent of shorter tree.
- The root of every tree holds also holds the **height** of the tree.
- In case two trees has the same height, we choose the root of the first tree point to the root of second. And the tree height is increased by 1.

Algorithm 4 Union(x, y)

```
1:  $a = \text{Find-Set}(x)$ 
2:  $b = \text{Find-Set}(y)$ 
3: if  $a.\text{height} \leq b.\text{height}$  then
4:   if  $a.\text{height} = b.\text{height}$  then
5:      $b.\text{height}++$ 
6:   end if
7:    $a.\text{parent} = b$ 
8: else
9:    $b.\text{parent} = a$ 
10: end if
```

Lemma

For any root x of a tree, let $\text{size}(x)$ be the number of nodes and $h(x)$ be the height of the tree. We have $\text{size}(x) \geq 2^{h(x)}$

Union by Height III

Proof.

(By induction)

- 1 At beginning, $h(x) = 0$, and $size(x) = 1$. We have $1 \geq 2^0 = 1$.
- 2 Suppose the assumption is true for any x and y before $Union(x, y)$. Let the size and height of the resulting tree be $size(x')$, and $h(x')$

- $h(x) < h(y)$, we have

$$size(x') = size(x) + size(y) \geq 2^{h(x)} + 2^{h(y)} \geq 2^{h(y)} = 2^{h(x')}$$

- $h(x) = h(y)$, we have

$$size(x') = size(x) + size(y) \geq 2^{h(x)} + 2^{h(y)} = 2^{h(y)+1} = 2^{h(x')}$$

- $h(x) > h(y)$, it is similar to the first case



Union by Height IV

Lemma

For n items, the running time of

- *Create-Set is $\mathcal{O}(1)$*
- *Find-Set is $\mathcal{O}(\log n)$*
- *Union is $\mathcal{O}(\log n)$*

respectively.

Proof.

- Obviously, $\text{Create-Set}(x)$ is $\mathcal{O}(1)$, and the running time of $\text{Union}(x, y)$ depends on $\text{Find-Set}(x)$.
- Since the running time of $\text{Find-Set}(x)$ depends on the height of the tree. From previous lemma, for any tree, we have

$$n \geq 2^h \Rightarrow h \leq \log n \Rightarrow h = \mathcal{O}(\log n)$$

Hence $\text{Find-Set}(x) = \mathcal{O}(\log n)$.

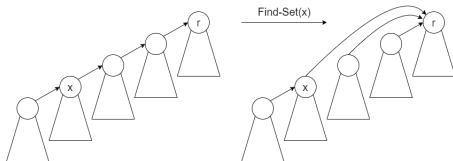


- 1 Kruskal's MST algorithm
 - The idea
 - The algorithm
 - Correctness

- 2 The Disjoint Set Union-Find data structure
 - The basic implementation
 - An improvement

Path Compression I

- We can make the running time even faster if we add another trick.
- In the $\text{Find-Set}(x)$, we trace the **path** from x to the root.
- Let r be the root of the tree, and the path from x to r is $x a_1 a_2 \dots a_k r$.
- As a by-product, we also make all the parent pointers of $x, a_1, a_2, \dots, a_k$ pointing to r **directly**.
 - Shortens the time of some future calls to Find-Set .
 - Makes the tree “flat”.



- This idea is called **path compression**.

Path Compression II

It is expected that in some **sequence** Find-Set operation, the running time is expected to be faster.

Question

By how much?

$\lg^{(i)} n$: defined recursively for nonnegative integers i as

$$\lg^{(i)} n = \begin{cases} n & n \text{ if } i = 0 \\ \lg(\lg^{(i-1)} n) & n \text{ if } i > 0 \text{ and } \lg^{(i-1)} n > 0 \\ \text{undefined} & n \text{ if } i > 0 \text{ and } \lg^{(i-1)} n \leq 0 \text{ or } \lg^{(i-1)} n \text{ is undefined} \end{cases}$$

The **iterated logarithm** is defined as

$$\lg^* n = \min\{i \geq 0, \lg^{(i)} n \leq 1\}$$

- a **very slow growing function**.
- e.g., $\lg^* 2 = 1$, $\lg^* 4 = 2$, $\lg^* 16 = 3$, $\lg^* 65536 = 4$,
 $\lg^* 2^{65536} = 5$.

The following theorem is stated without proof.

Theorem

A sequence of m Create-Set, Find-Set and Union operations, n of which are Create-Set operations, can be performed on a disjoint-set forest with union by height and path compression in worst-case time $\mathcal{O}(m \lg^ n)$*

Question

What is the running time of Kruskal's algorithm if we employ this implementation of disjoint set Union-Find?